Mini INS (Inertial Naviation System) en ROS2 (Paso a paso completo en python para crear un nodo)

🎯 Objetivo

Crear un nodo que:

1. Probar ROS2 **[Jazzy/Humble]** y Rviz en Ubuntu.
2. Reciba datos de una IMU (Unidad de medición inercial) (/imu)
3. Integrar velocidad angular
4. Publicar orientación como **TF (cuaternión)**

🔧 PASO 0: Preparar entorno

1. Cargar ROS2 [Jazzy/Humble] en un terminal

source /opt/ros/**[jazzy/humble]**/setup.bash

2. Crear workspace

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
colcon build
source install/setup.bash
```

📦 PASO 1: Crear paquete ROS2

```
cd ~/ros2_ws/src

ros2 pkg create imu_orientation \
 --build-type ament_python \
 --dependencies rclpy sensor_msgs geometry_msgs tf2_ros
```

📁 Estructura esperada

```
imu_orientation/
├── imu_orientation/
│   └── __init__.py
├── package.xml
├── setup.py
└── setup.cfg
```

🧠 PASO 2: Crear nodo integrador

📄 Crear archivo:

```
cd imu_orientation/imu_orientation
nano imu_integrator.py
```

🖍️ Código completo (versión robusta)

```
import rclpy
from rclpy.node import Node

from sensor_msgs.msg import Imu
from geometry_msgs.msg import TransformStamped
from tf2_ros import TransformBroadcaster

from scipy.spatial.transform import Rotation as R
import numpy as np


class IMUIntegrator(Node):

    def __init__(self):
        super().__init__('imu_integrator')

        # Suscriptor IMU
        self.sub = self.create_subscription(
            Imu,
            '/imu',
            self.imu_callback,
            10
        )

        # TF broadcaster
        self.br = TransformBroadcaster(self)

        # Estado (cuaternión)
        self.q = R.identity()

        # Tiempo
        self.last_time = None

        self.get_logger().info("IMU Integrator iniciado")

    def imu_callback(self, msg):

        current_time = self.get_clock().now()

        # Primera iteración
        if self.last_time is None:
            self.last_time = current_time
```

```
            return

        # Delta de tiempo
        dt = (current_time - self.last_time).nanoseconds * 1e-9
        self.last_time = current_time

        # Evitar dt inválido
        if dt <= 0:
            return

        # Velocidad angular (rad/s)
        w = np.array([
            msg.angular_velocity.x,
            msg.angular_velocity.y,
            msg.angular_velocity.z
        ])

        # Integración (cuaternión incremental)
        dq = R.from_rotvec(w * dt)

        # Actualizar orientación
        self.q = self.q * dq

        # Normalización (importante)
        self.q = self.q.normalized()

        # Publicar TF
        self.publish_tf(current_time)

    def publish_tf(self, current_time):

        t = TransformStamped()

        t.header.stamp = current_time.to_msg()
        t.header.frame_id = 'world'
        t.child_frame_id = 'imu_link'

        # Sin traslación (solo orientación)
        t.transform.translation.x = 0.0
        t.transform.translation.y = 0.0
        t.transform.translation.z = 0.0

        q = self.q.as_quat()

        t.transform.rotation.x = float(q[0])
        t.transform.rotation.y = float(q[1])
        t.transform.rotation.z = float(q[2])
        t.transform.rotation.w = float(q[3])
```

```
        self.br.sendTransform(t)


def main():
    rclpy.init()
    node = IMUIntegrator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

⚙️ **PASO 3: Registrar el nodo**

Editar setup.py:

nano ~/ros2_ws/src/imu_orientation/setup.py

Agregar o editar:

```
entry_points={
    'console_scripts': [
        'imu_integrator = imu_orientation.imu_integrator:main',
    ],
},
```

🔨 **PASO 4: Compilar**

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

📡 **PASO 5: Generar datos de IMU (simulados)**

Como no tienes IMU real, creamos un nodo simulador.

📄 **Crear:**

cd ~/ros2_ws/src/imu_orientation/imu_orientation

nano imu_simulator.py

🖍️ **Código**

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Imu
```

```
class IMUSimulator(Node):

    def __init__(self):
        super().__init__('imu_simulator')

        self.pub = self.create_publisher(Imu, '/imu', 10)
        self.timer = self.create_timer(0.01, self.publish_imu)

    def publish_imu(self):

        msg = Imu()

        # Rotación constante en Z
        msg.angular_velocity.z = 0.5  # rad/s

        self.pub.publish(msg)


def main():
    rclpy.init()
    node = IMUSimulator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

Registrar en setup.py

```
'imu_simulator = imu_orientation.imu_simulator:main',
```

Recompilar

```
colcon build
source install/setup.bash
```

▶️ **PASO 6: Ejecutar sistema**

Terminal 1:

```
source /opt/ros/[jazzy/humble]/setup.bash

source install/setup.bash

ros2 run imu_orientation imu_simulator
```

Terminal 2:

source /opt/ros/**[jazzy/humble]**/setup.bash

source install/setup.bash

ros2 run imu_orientation imu_integrator

👀 **PASO 7: Visualizar en Rviz. (Ver anexo 1)**

En un terminal, ejecutar:

rviz2

Agregar:

- TF
- Axes

🎯 **Resultado esperado**

Verás:

- Frame imu_link rotando continuamente sobre eje Z

🔍 **PASO 8: Verificar TF (requiera un visor de pdf para terminal, como zathura)**

ros2 run tf2_tools view_frames

Permite ver la manera en la que los marcos de referencia están relacionados.

EJERCICIOS:

🧠 **PASO 9: Validación conceptual**

Comprueba:

- Si duplicas angular_velocity.z → rota más rápido?
- Si cambias eje → cambia eje de rotación?

⚠️ **ERRORES COMUNES (muy importante)**

❌ **1. No normalizar cuaternión**

→ deriva numérica

❌ **2. dt incorrecto**

→ rotaciones erráticas

❌ **3. No hacer source en el terminal, al iniciar.**

source install/setup.bash

🚀 **EXTENSIONES (recomendadas)**

Realizar:

- 🔹 **1. Movimiento en 3 ejes**

msg.angular_velocity.x = 0.2
msg.angular_velocity.y = 0.3
msg.angular_velocity.z = 0.5

- 🔹 **2. Comparar con Euler (error conceptual)**
- 🔹 **3. Agregar traslación (odometría)**

ANEXO 1. DETALLES SOBRE EL PASO 7 de visualización en Rviz2

🧭 **PASO 7: Visualización en RViz2 (TF + Axes)**

🚀 **1. Abrir RViz2**

En una terminal:

rviz2

Se abrirá la interfaz gráfica.

🧩 **2. Entender la interfaz (rápido)**

- Panel izquierdo → Displays (visualizaciones)
- Panel derecho → Propiedades
- Parte superior → Fixed Frame

⚠️ **3. Configurar el "Fixed Frame" (MUY IMPORTANTE)**

En la parte superior izquierda:

👉 Cambia:

Fixed Frame → world

🧠 **¿Por qué?**

Tu nodo publica:

```
t.header.frame_id = 'world'
```

Si no coinciden → **no verás nada**

🔄 **4. Agregar TF**

🔹 **Paso a paso:**

1. En el panel izquierdo:
   👉 Click en **"Add"**
2. Se abre ventana → selecciona:
   👉 **TF**
3. Click en **OK**

✅ **¿Qué hace TF?**

- Muestra todos los frames del sistema.
- Dibuja:
    - ejes de coordenadas
    - relaciones entre frames

⚙️ **Configuración recomendada (TF)**

En el panel derecho:

- ✓ Show Axes → ON
- ✓ Show Names → ON
- ✓ Marker Scale → 1.0 (o ajusta)

📍 **5. Agregar Axes (para ver un frame específico)**

🔹 **Paso a paso:**

1. Click en **Add**
2. Selecciona:
   👉 **Axes**
3. OK

⚙️ **Configurar Axes**

En el panel derecho:

- **Reference Frame:**

imu_link

🎯 **Resultado esperado**

Verás:

- Un sistema de ejes (X rojo, Y verde, Z azul) rotando continuamente

---

🎨 **6. Qué deberías ver**

- Frame world (fijo)
- Frame imu_link (girando)

🔍 **7. Si NO ves nada (debug rápido)**

✓ **Verifica TF activo en un terminal que tenga el source:**

ros2 topic echo /tf

Debe mostrar mensajes.

✓ **Verifica nombres de frames:**

ros2 run tf2_tools view_frames

✓ **Verifica Fixed Frame:**

Debe ser EXACTAMENTE:

world

✓ **Verifica nodo corriendo:**

ros2 node list

Debe aparecer:

- /imu_integrator
- /imu_simulator

⚠️ **ERRORES COMUNES**

❌ **1. Fixed Frame incorrecto**

→ No aparece nada

❌ **2. No agregar TF**

→ No ves frames

❌ **3. Nombre mal escrito**

Ej:

imu-link ❌

imu_link ✅

💡 **EJERCICIO:**

Ensayar:

🔹 **Cambiar eje de rotación**

msg.angular_velocity.x = 0.5

👉 Verás cambio inmediato en RViz

🧠 **Conexión conceptual**

Lo que estás viendo en RViz es:

- ✓ Pose dependiente del tiempo
- ✓ Integración de velocidad angular
- ✓ Transformación entre frames